GO PROGRAMMING MASTER COURSE

Week 12, Day 1

Mutex, WaitGroup, context, and races — make shared memory and cancellation safe.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

SINCE LAST WEEK

You have a pool. Now protect shared state.

- Worker pool + WaitGroup join — keep that.
- Week 10 previewed `go test -race`; today we **fix** the red → green loop.
- Create still sync. Import still uses a pool. Map must stay locked.

TODAY

If Day 1 of week 12 works, you can

- Protect a shared counter/map with `sync.Mutex` (and name `RWMutex`).
- Use WaitGroup as a join (not a lock).
- Pass `c.Request.Context()` into repository signatures.
- Demo a timeout with `context.WithTimeout`.
- Sketch graceful shutdown with `signal.Notify`.
- Get `go test -race` green on the lab.

SOFTWARE ENGINEERING

Shared memory needs a rule

Channels coordinate work.
Mutexes protect *data*.
Context cancels *work*.

- SMS: lock the in-memory map; add `context.Context` to repo methods (may ignore until week 14).
- Homework: README test command includes `-race`.

DATA RACE

Two goroutines · one variable · no sync

```
var n int
go func() { n++
go func() { n++
```

- May “work” until it doesn’t — undefined behavior.
- `go test -race` makes it visible (needs cgo + gcc).

MUTEX

One writer (or reader) at a time

```
var mu sync.Mutex
mu.Lock()
n++
mu.Unlock()
// prefer: defer mu.Unlock()
```

- Hold locks short. Don’t call slow I/O under lock.
- Same mutex for the same data — document ownership.

RWMUTEX

Many readers · one writer

```
var mu sync.RWMutex
mu.RLock(); defer mu.RUnloc
mu.Lock();  defer mu.Unlock
```

- Use when reads dominate and critical sections are tiny.
- SMS starter: `Mutex` is enough; mention RWMutex so the word sticks.

WAITGROUP

Join, not lock

```
var wg sync.WaitGroup
wg.Add(N)
go func() { defer wg.Done(); wor
wg.Wait() // all Done
```

- Does **not** protect n++ — still need a mutex (or channels).
- Week 11: WaitGroup before closing results.

CONTEXT

Cancel · deadline · timeout

```
ctx, cancel := context.WithTimeout
defer cancel()
select {
case <-ctx.Done():
    return ctx.Err() // deadline e
case result := <-work:
    return result, nil
}
```

- Context flows down the call stack — first param by convention.
- Always `defer cancel()` when you create a derived context.

GIN

c.Request.Context() on every handler

```
func (h *StudentHandler)
    ctx := c.Request.Cont
    s, err := h.repo.GetB
     ...
}
```

- Client disconnect / server shutdown cancels that context.
- Repo may ignore `ctx` until Postgres (week 14) — signature still changes now.

SMS MILESTONE

Lock the map · pass context

```
func (r *StudentRepository) Create(ctx
    _ = ctx // ignored until DB
    r.mu.Lock()
    defer r.mu.Unlock()
     ...
}
```

- Handlers pass `c.Request.Context()`.
- Demo timeout endpoint: slow work respects deadline.

GRACEFUL SHUTDOWN

Sketch — don’t leave listeners hanging

```
ctx, stop := signal.NotifyContext(contex
defer stop()
srv := &http.Server{Addr: addr, Handler:
go srv.ListenAndServe()
<-ctx.Done()
srv.Shutdown(context.WithTimeout( ... ))
```

- Stops accepting; drains in-flight requests.
- Enough of a sketch — polish later with Postgres.

LAB

Race detector: red → mutex → green

```
cd starter/race-lab
go test -race ./counter/
# fix with Mutex
go test -race ./...
```

- Needs `CGO_ENABLED=1` and a C compiler (`build-essential` on Ubuntu).

LIVE DEMO -RACE

Show the FAIL, then the fix

```
go test -race ./counter/
# DATA RACE
# after SafeAdd with Mutex:
go test -race ./counter/   # ok
```

LIVE DEMO

CONTEXT TIMEOUT

Demo endpoint respects deadline

```
GET /api/demo/slow?ms=3000
# with 1s server timeout → 504 / ca
```

LIVE DEMO

SHUTDOWN

Ctrl+C drains, then exits

```
go run ./cmd/server
# Ctrl+C → "shutting down" → clean exit
```

PITFALLS

Common week-12 bugs

- WaitGroup without mutex → still races.
- Forget `Unlock` / unlock wrong mutex → deadlock.
- Create context without `defer cancel()`.
- Ignore `ctx.Done()` in long loops.
- `-race` fails: no gcc / CGO disabled.

LAB · REST OF CLASS

Done when all of this is true

- `go test -race ./...` green on race-lab
- Can explain Mutex vs WaitGroup vs channel
- Repo methods take `context.Context`; handlers pass request context
- Timeout demo works (or sketched)
- Optional: graceful shutdown in `main`

HOMEWORK

PR with `-race` in the README

- Test command documents `go test -race ./...`
- Mutex on shared maps; context on repo signatures.
- PR notes: what was red, what made it green.

NEXT SESSION

Week 13 — testing and code quality

- Table-driven tests, httptest, fakes.
- CI: gofmt, vet, test.
- Bring a green `-race` habit.

WORDS TO KEEP

Say these correctly next week

Mutex

Exclusive access to shared memory.

WaitGroup

Join N goroutines; not a lock.

context

Cancel / deadline flowing down the stack.

-race

Detector for concurrent unsynchronized access.

LAYOUT REMINDER

Where lab code lives

```
docs/week-12-day-01/starter/
  race-lab/                  # red → mutex → green
  backend/
    internal/repository/     # Mutex + context.Context pa
    cmd/server/main.go       # graceful shutdown sketch
    internal/handlers/       # c.Request.Context() + time
```

HONEST LIMITS

What this week is not

- Not Postgres / real query cancel (week 14).
- Not deep testing / CI (week 13).
- Not “channels instead of every mutex.”
- Not production-hardening every edge case.

SAFETY

Still true

- Cap demo timeouts; don’t sleep forever in handlers.
- No secrets in fixtures.
- `gofmt`; no binaries in git.

REVIEW BAR

Instructor merges when

- `go test -race ./...` documented and green (or lab evidence)
- Shared maps locked
- Repo signatures take context; handlers pass request context
- Create/list/import still behave

Questions